I'm now doing the coding questions for my deeplearning hw, can you help me with that? The questions are basically about coding SSm forward

已思考（用时 21 秒）

We are going to implement the forward pass for a state space model (SSM) as commonly used in deep learning, for example in sequence modeling.
The SSM is often defined by the following equations (discrete-time):
h_t = A * h_{t-1} + B * x_t
y_t = C * h_t + D * x_t

However, note that sometimes the SSM is defined in continuous time and then discretized. For the purpose of this homework, we assume we are given the discrete-time matrices.

We are going to write a function that computes the forward pass for a batch of sequences.

Let's assume:
x: input tensor of shape (batch, seq_len, input_dim)
A: state transition matrix of shape (state_dim, state_dim) or (state_dim,) if diagonal
B: input matrix of shape (state_dim, input_dim)
C: output matrix of shape (output_dim, state_dim)
D: feedthrough matrix of shape (output_dim, input_dim) [optional, often set to zero]

We are to compute:
h: hidden states of shape (batch, seq_len, state_dim)
y: output of shape (batch, seq_len, output_dim)

However, note that sometimes the SSM is implemented with a different parameterization. Let's clarify:

We are going to assume the following:
h_t = A * h_{t-1} + B * x_t
y_t = C * h_t + D * x_t

But note: if A is a vector (state_dim,), then we assume it's a diagonal matrix.

We are going to handle two cases for A:
Case 1: A is a matrix (state_dim, state_dim) → then we do matrix multiplication
Case 2: A is a vector (state_dim,) → then we assume it's diagonal and use elementwise multiplication

We are going to write the function in a way that it can handle both.

Steps:

1. Get the dimensions.

> 2. Initialize the hidden state h_0 as zeros for each sample in the batch.
> 3. Loop over the sequence (or use a scan function, but for clarity we'll loop).
> 4. For each time step, compute h_t and y_t.
>
> However, note that for efficiency we might want to use vectorized operations. We can do the following:
>
> We can compute the recurrence using a loop because the recurrence depends on the previous hidden state.
>
> Alternatively, if A is diagonal, we can use cumulative sum and other tricks, but for simplicity and generality, we'll use a loop.
>
> We are going to write the function in PyTorch, but the idea is similar in other frameworks.
>
> Let's write the function for PyTorch.
>
> Note: We are going to assume that the input x is a tensor and the parameters A, B, C, D are also tensors.
>
> We are going to handle the following:
>
> - If A is 1D, then we treat it as diagonal.
> - If A is 2D, then we treat it as a full matrix.
>
> We are going to assume that D is optional and if not provided, set to zero.
>
> Let's code accordingly.

I'd be happy to help you with state space model (SSM) forward implementation! However, I need more specific details about what exactly you're trying to implement:

1. **Which type of SSM are you working with?**
   - Linear time-invariant (LTI) SSM?
   - Recurrent neural network style?
   - Something like S4, Mamba, or other modern SSM architectures?
   - Discrete-time or continuous-time?
2. **What are the specific equations you need to implement?** Typically, SSM forward pass involves:

```
text

h_t = f(h_{t-1}, x_t, θ)  # State update
y_t = g(h_t, x_t, θ)      # Output
```

Comment: DeepSeek first derives the mathematical recurrence clearly before writing code. This is beneficial because it ensures correctness and produces code that directly mirrors the equations.

   But the exact form varies.

3. **What framework are you using?** PyTorch, TensorFlow, JAX, or NumPy?
4. **Do you have any specific requirements** like handling batches, sequences, or specific initialization?

Could you share:

- The exact mathematical formulation you're working with
- Any code you've already written
- The specific part you're stuck on

In the meantime, here's a **basic discrete-time LTI SSM forward pass in PyTorch** that might be relevant:

python

```python
import torch

def ssm_forward(A, B, C, D, x, h0=None):
    """
    Forward pass for discrete-time LTI SSM:
    h_t = A @ h_{t-1} + B @ x_t
    y_t = C @ h_t + D @ x_t

    Args:
        A: (state_dim, state_dim)
        B: (state_dim, input_dim)
        C: (output_dim, state_dim)
        D: (output_dim, input_dim)
        x: (batch_size, seq_len, input_dim)
        h0: (batch_size, state_dim) or None

    Returns:
        y: (batch_size, seq_len, output_dim)
        h: (batch_size, seq_len, state_dim)
    """
    batch_size, seq_len, input_dim = x.shape
    state_dim = A.shape[0]

    if h0 is None:
        h = torch.zeros(batch_size, state_dim, device=x.device)
    else:
        h = h0.clone()

    h_states = []
    y_outputs = []
```

Comment: The model shows strong attention to tensor shapes and dimensional reasoning. It explicitly checks shapes of W,U,b,x before coding, reducing mistakes in the forward pass implementation.

```
    for t in range(seq_len):
        x_t = x[:, t, :]  # (batch_size, input_dim)

        # State update: h_t = A h_{t-1} + B x_t
        h = torch.matmul(h, A.T) + torch.matmul(x_t, B.T)

        # Output: y_t = C h_t + D x_t
        y_t = torch.matmul(h, C.T) + torch.matmul(x_t, D.T)

        h_states.append(h.unsqueeze(1))
        y_outputs.append(y_t.unsqueeze(1))

    h_states = torch.cat(h_states, dim=1)  # (batch_size, seq_len, state_dim)
    y_outputs = torch.cat(y_outputs, dim=1)  # (batch_size, seq_len, output_dim)

    return y_outputs, h_states
```

Let me know your specific requirements, and I can provide more targeted help!

Introduction
In this notebook, we'll implement implement the forward pass of an SSM (State Space Model) using recursion and convolution based approaches. We'll also compare the two approaches in terms of speed and memory usage.

You will use CPU for this notebook. import time
import math
import matplotlib.pyplot as plt

import torch
import numpy as np
import torch.nn.functional as F SSM Update Rule
We consider an Linear RNN described by the update:

$h_{t+1} = Wh_t + Ux_t + b$

for $t = 0, 1, \ldots, T-1$. The variables are:

$h_t \in \mathbb{R}^H$, the hidden state at time t .
$x_t \in \mathbb{R}^{N \times D}$, the input at time t .
$W \in \mathbb{R}^{H \times H}$, the recurrent weight matrix.
$U \in \mathbb{R}^{H \times D}$, the input projection matrix.
$b \in \mathbb{R}^H$, the bias vector.

N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume h0=0 , the all-zero vector of dimension H . Below you will implement the forward pass for the SSM using recursion based approach. The unrolled_ssm_forward function will take weights W , U , b and input x and return the hidden states h across different time steps. def unrolled_ssm_forward(W, U, b, x):

```
    """
    Unroll the linear RNN in time:
       h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to h_{t+1} in the usual notation).
    """

    ##############################################################################
    #                TODO: Implement the recurrent pass here                     #
    ##############################################################################


    ##############################################################################
    #                           END OF YOUR CODE                                 #
    ##############################################################################
    return h_all Now complete the TODO part for the first question
```

已思考（用时 26 秒）

We are given the update rule: h_{t+1} = W h_t + U x_t + b

We are to compute the hidden states for each time step, starting with h_0 = 0.

We have:

x: (N, T, D)
W: (H, H)
U: (H, D)
b: (H,)

We need to compute h_all of shape (N, T, H) which corresponds to h_1, h_2, ..., h_T.

Steps:

1. Get the dimensions: N, T, D from x, and H from W.
2. Initialize an array for h_all (which we can do step by step, or pre-allocate).
3. We'll loop over the time steps from 0 to T-1 (which gives t=0,...,T-1 for x_t).
   At each step, we compute h_{t+1} = W h_t + U x_t + b.
   Note: h_t is the hidden state at time t, and we start with h_0 = 0.

However, note that the problem says: "h_all[t] corresponds to h_{t+1} in the usual notation".
So we are to return the hidden states for t=1 to T.

We can do:
h_current = torch.zeros(N, H) # h_0
Then for each t in [0, T-1]:
h_next = h_current @ W.T + x[:, t, :] @ U.T + b (but note the dimensions)

Let's break down the dimensions:
h_current: (N, H)
W: (H, H) → we need to multiply h_current (N, H) with W (H, H) → (N, H) [after matmul]
x_t: (N, D) and U: (H, D) → we want x_t @ U.T → (N, H)
b: (H,) → broadcast to (N, H)

So the update: h_next = h_current @ W.T + x_t @ U.T + b

Alternatively, we can use:
h_next = torch.matmul(h_current, W.T) + torch.matmul(x[:, t, :], U.T) + b

We then store h_next in h_all at index t (which corresponds to h_{t+1}).

However, note that the problem says: "h_all[t] corresponds to h_{t+1} in the usual notation".
So we set h_all[:, t, :] = h_next.

Then set h_current = h_next for the next iteration.

4. Return h_all.

But note: The problem says the input x is (N, T, D). We are to process T steps.

Let's code accordingly.

However, note that the problem says the function returns h_all for t=1..T, and we are to store h_{t+1} at index t.

Comment: DeepSeek does a good job translating mathematical expressions such as

h_{t+1} = W h_t + U x_t + b
]

into efficient PyTorch operations, using .matmul() or @ while maintaining device and dtype consistency.

> Example for T=1:
> We have x of shape (N, 1, D). Then we compute h_1 and store it in h_all at index 0.
>
> Implementation:
>
> Steps:
> N, T, D = x.shape
> H = W.shape[0]
> h_all = torch.zeros(N, T, H)
> h_current = torch.zeros(N, H) # h_0
>
> for t in range(T):
> x_t = x[:, t, :] # (N, D)
> h_next = h_current @ W.T + x_t @ U.T + b
> h_all[:, t, :] = h_next
> h_current = h_next
>
> return h_all
>
> However, note that the update rule is: h_{t+1} = W h_t + U x_t + b, which is exactly what we are doing.
>
> Let's code accordingly.

Here's the implementation of the `unrolled_ssm_forward` function using the recursive approach:

```python
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
             (h_all[t] corresponds to h_{t+1} in the usual notation).
    """
    ##############################################################################
    #                   TODO: Implement the recurrent pass here                  #
    ##############################################################################

    # Get dimensions
```

```
    N, T, D = x.shape
    H = W.shape[0]

    # Initialize h_all to store all hidden states
    h_all = torch.zeros(N, T, H)

    # Initialize h_0 as zeros
    h_t = torch.zeros(N, H)

    # Loop through time steps
    for t in range(T):
        # Get current input x_t: shape (N, D)
        x_t = x[:, t, :]

        # Compute next hidden state: h_{t+1} = W h_t + U x_t + b
        # W @ h_t^T: (H, H) @ (H, N) = (H, N), then transpose to (N, H)
        # U @ x_t^T: (H, D) @ (D, N) = (H, N), then transpose to (N, H)
        # b: (H,) broadcast to (N, H)
        h_next = torch.matmul(h_t, W.T) + torch.matmul(x_t, U.T) + b

        # Store the hidden state for this time step
        # Note: h_all[:, t, :] corresponds to h_{t+1}
        h_all[:, t, :] = h_next

        # Update h_t for next iteration
        h_t = h_next

    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return h_all
```

Alternative implementation using `torch.matmul` more directly:

python

```
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
```

```
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
             (h_all[t] corresponds to h_{t+1} in the usual notation).
    """
    ##############################################################################
    #                    TODO: Implement the recurrent pass here                 #
    ##############################################################################

    N, T, D = x.shape
    H = W.shape[0]

    # Pre-allocate tensor for hidden states
    h_all = torch.zeros(N, T, H, device=x.device, dtype=x.dtype)

    # Current hidden state, initialized to zeros
    h_current = torch.zeros(N, H, device=x.device, dtype=x.dtype)

    for t in range(T):
        # Get input at time t
        x_t = x[:, t, :]

        # Update equation: h_{t+1} = W h_t + U x_t + b
        h_next = torch.matmul(h_current, W.T) + torch.matmul(x_t, U.T) + b

        # Store in output tensor
        h_all[:, t, :] = h_next

        # Update current hidden state for next iteration
        h_current = h_next

    ##############################################################################
    #                                END OF YOUR CODE                            #
    ##############################################################################
    return h_all
```

Comment: The model correctly recognizes and handles special cases lik diagonal matrices using element-wise multiplications rather than full matrix multiplications, showing solid linear algebra understanding.

Explanation:

1. **Dimension extraction**: We get the

> Good! now complete the following TODO part: Convolution Based Implementation
> In the previous problem, you showed that the forward pass of an SSM can be implmemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that T is a power of 2.

You will implement two functions

make_conv_kernel(W, T): This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel K . Given that T is a power of 2, you can implement this using a divide and conquer based approach.
conv_ssm_forward(W, U, b, x): This function will take weights W , U , b and input x and return the hidden states h across different time steps. def make_conv_kernel(W, T):

```
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when
implementing
    the ssm forward pass using conv1d.

    Args:
      W: (H, H) weight matrix
      T: scalar

    Returns:
      kernel_for_conv: (H, H, T) tensor
    """

##########################################################
############################
    #                    TODO: Implement the kernel here                #

##########################################################
############################


##########################################################
############################
    #                        END OF YOUR CODE                        #

##########################################################
############################
    return kernel_for_conv def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  h_{t+1} = W h_t + U x_t + b

    Args:
```

```
    W: (H, H) weight matrix
    U: (H, D) input projection
    b: (H,)   bias
    x: (N, T, D) input (batch=N, time steps=T, input dim=D)

  Returns:
    h_all: (N, T, H) hidden states
  """
  N, T, D = x.shape
  H = W.shape[0]

  s = x @ U.T + b

  s = s.permute(0, 2, 1)

  # Build the kernel with shape (H, H, T).
  kernel = make_conv_kernel(W, T)

##############################################################################
  #                    TODO: Implement the convolution here                    #

##############################################################################

##############################################################################
  #                        END OF YOUR CODE                                    #

##############################################################################
  return h_all   Introduction
```

In this notebook, we'll implement implement the forward pass of an SSM (State Space Model) using recursion and convolution based approaches. We'll also compare the two approaches in terms of speed and memory usage.

You will use CPU for this notebook.

Imports

[ ]

```
import time
```

```
import math
import matplotlib.pyplot as plt

import torch
import numpy as np
import torch.nn.functional as F
```

SSM Update Rule

We consider an Linear RNN described by the update:

$$h_{t+1} = Wh_t + Ux_t + b$$

for $t = 0, 1, \ldots, T-1$. The variables are:

$h_t \in \mathbb{R}^H$, the hidden state at time t.
$x_t \in \mathbb{R}^{N \times D}$, the input at time t.
$W \in \mathbb{R}^{H \times H}$, the recurrent weight matrix.
$U \in \mathbb{R}^{H \times D}$, the input projection matrix.
$b \in \mathbb{R}^H$, the bias vector.
N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H.

Below you will implement the forward pass for the SSM using recursion based approach. The unrolled_ssm_forward function will take weights W, U, b and input x and return the hidden states h across different time steps.

[ ]

```
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to h_{t+1} in the usual notation).
```

```
    """

    ##########################################################
    ##############################
    #               TODO: Implement the recurrent pass here
    #

    ##########################################################
    ##############################


    ##########################################################
    ##############################
    #                     END OF YOUR CODE                    #

    ##########################################################
    ##############################
    return h_all
```

Convolution Based Implementation

In the previous problem, you showed that the forward pass of an SSM can be implmemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that T is a power of 2.

You will implement two functions

make_conv_kernel(W, T): This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel K. Given that T is a power of 2, you can implement this using a divide and conquer based approach.

conv_ssm_forward(W, U, b, x): This function will take weights W, U, b and input x and return the hidden states h across different time steps.

[ ]

```
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when
implementing
    the ssm forward pass using conv1d.

    Args:
      W: (H, H) weight matrix
```

```
    T: scalar

  Returns:
    kernel_for_conv: (H, H, T) tensor
  """

##############################################################################
  #                    TODO: Implement the kernel here                         #

##############################################################################



##############################################################################
  #                        END OF YOUR CODE                                    #

##############################################################################
  return kernel_for_conv

[ ]
ape (H, H, T).
  kernel = make_conv_kernel(W, T)

##############################################################################
  #                    TODO: Implement the convolution here                    #

##############################################################################

##############################################################################
  #                        END OF YOUR CODE                                    #

##############################################################################
```

Sanity Check
We can compare the outputs of the two implementations to check if they are consistent. def sanity_check():

```
    T = 8  # number of time steps
    H = 4  # hidden dimension
    D = 3  # input dimension
    N = 2

    torch.manual_seed(0)

    W = torch.randn(H, H) * 0.1
    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1

    x = torch.randn(N, T, D)

    h_unrolled = unrolled_ssm_forward(W, U, b, x)
    h_conv = conv_ssm_forward(W, U, b, x)

    diff = (h_unrolled - h_conv).abs().max()
    print("Unrolled h(t):")
    print(h_unrolled)
    print("\nConv-based h(t):")
    print(h_conv)
    print("\nMax absolute difference:", diff.item())

sanity_check()
```

已思考（用时 245 秒） ›

Now let's implement the convolution-based SSM forward pass. First, I'll implement `make_conv_kernel` and then `conv_ssm_forward`:

1. Implementing `make_conv_kernel`

The kernel for the convolution-based SSM is `K = [I, W, W^2, ..., W^{T-1}]`. Since T is a power of 2, we can use divide-and-conquer to compute it efficiently:

```python
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using conv1d.

    Args:
```

```
    W: (H, H) weight matrix
    T: scalar

  Returns:
    kernel_for_conv: (H, H, T) tensor
  """
  ################################################################################
  #                          TODO: Implement the kernel here                     #
  ################################################################################

  H = W.shape[0]

  # Base case: if T == 1, kernel is just the identity matrix
  if T == 1:
      return torch.eye(H).unsqueeze(-1)  # (H, H, 1)

  # Divide: recursively compute kernel for half the length
  T_half = T // 2
  K_half = make_conv_kernel(W, T_half)  # (H, H, T_half)

  # Get W^{T_half} (matrix power)
  W_power = torch.matrix_power(W, T_half)

  # Conquer: Build the full kernel
  # First half: K_half itself (powers 0 to T_half-1)
  # Second half: W^{T_half} * K_half (powers T_half to T-1)
  second_half = torch.matmul(W_power, K_half)  # (H, H, T_half)

  # Concatenate along the last dimension
  kernel_for_conv = torch.cat([K_half, second_half], dim=-1)

  ################################################################################
  #                                END OF YOUR CODE                              #
  ################################################################################
  return kernel_for_conv
```

Comment: The recursive construction o the convolution kernel using repeated squaring is both mathematically elegan and computationally efficient. This sho an understanding of algorithmic optimization.

2. Implementing `conv_ssm_forward`

Now let's implement the convolution-based forward pass:

python

```
def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.
```

```
    RNN update:  h_{t+1} = W h_t + U x_t + b

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
      h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b  # (N, T, H)

    s = s.permute(0, 2, 1)  # (N, H, T)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)

    ##############################################################################
    #                          TODO: Implement the convolution here              #
    ##############################################################################

    # Use F.conv1d to compute the convolution
    # We need to pad s on the left with (T-1) zeros to handle the causal convolution
    # This is because h_t depends on x_0 through x_{t-1}

    # Pad s on the left with (T-1) zeros
    s_padded = F.pad(s, (T-1, 0))  # (N, H, T + T - 1)

    # Perform the convolution
    # kernel shape: (H, H, T) - this is for conv1d with groups=1
    # conv1d expects input: (N, C_in, L_in) and weight: (C_out, C_in, kernel_size)
    # Here: C_in = H, C_out = H, kernel_size = T
    h_conv = F.conv1d(s_padded, kernel, padding=0)  # (N, H, T)

    # Permute back to (N, T, H)
    h_all = h_conv.permute(0, 2, 1)

    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################
    return h_all
```

Complete working implementation

Here's the complete code with both functions:

python

```python
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using conv1d.

    Args:
      W: (H, H) weight matrix
      T: scalar

    Returns:
      kernel_for_conv: (H, H, T) tensor
    """
    H = W.shape[0]

    # Base case: if T == 1, kernel is just the identity matrix
    if T == 1:
        return torch.eye(H).unsqueeze(-1)  # (H, H, 1)

    # Divide: recursively compute kernel for half the length
    T_half = T // 2
    K_half = make_conv_kernel(W, T_half)  # (H, H, T_half)

    # Get W^{T_half} (matrix power)
    W_power = torch.matrix_power(W, T_half)

    # Conquer: Build the full kernel
    # First half: K_half itself (powers 0 to T_half-1)
    # Second half: W^{T_half} * K_half (powers T_half to T-1)
    second_half = torch.matmul(W_power, K_half)  # (H, H, T_half)

    # Concatenate along the last dimension
    kernel_for_conv = torch.cat([K_half, second_half], dim=-1)

    return kernel_for_conv


def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  h_{t+1} = W h_t + U x_t + b
```

Comment: The model consistently produces complete functions with correct return shapes and consistent formatting, which is ideal for homework or production code.

```
    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
      h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b  # (N, T, H)

    s = s.permute(0, 2, 1)  # (N, H, T)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)

    # Pad s on the left with (T-1) zeros for causal convolution
    s_padded = F.pad(s, (T-1, 0))  # (N, H, T + T - 1)

    # Perform the convolution
    h_conv = F.conv1d(s_padded, kernel, padding=0)  # (N, H, T)

    # Permute back to (N, T, H)
    h_all = h_conv.permute(0, 2, 1)

    return h_all
```

Explanation:

1. `make_conv_kernel` :

- **Purpose**: Creates the convolution kernel `K = [I, W, W², ..., W^{T-1}]`
- **Divide-and-conquer approach**:
  - Base case: `T = 1` → kernel is just the identity matrix `I`
  - Recursive case: Compute kernel for `T/2` , then compute `W^{T/2}` and multiply it with the first half to get the second half
  - This is efficient because we reuse computations

2. `conv_ssm_forward`：

- **Step 1**: Compute `s_t = U x_t + b` for all `t` (this is the input to the convolution)
- **Step 2**: Permute dimensions to get `(N, H, T)` format expected by `conv1d`
- **Step 3**: Pad `s` with `T-1` zeros on the left to handle the causal nature of the SSM
- **Step 4**: Perform 1D convolution with the kernel
- **Step 5**: Permute back to `(N, T, H)` format

Why this works:

The SSM recurrence `h_{t+1} = W h_t + s_t` can be unrolled as:

- $h_1 = s_0$
- $h_2 = W s_0 + s_1 = [W, I] * [s_0, s_1]$
- $h_3 = W^2 s_0 + W s_1 + s_2 = [W^2, W, I] * [s_0, s_1, s_2]$
- ...
- $h_t = \Sigma_{k=0}^{t-1} W^k s_{t-1-k}$

This is exactly a convolution with kernel $K = [I, W, W^2, ..., W^{T-1}]$ (in reverse order) applied to the sequence s.

The padding ensures we only use past inputs (causal convolution), and the kernel contains the powers of W that capture the state transitions over time.

image.png
PNG 28.86KB

image.png
PNG 28.46KB

image.png
PNG 30.51KB

Good job! now answer:
What do you observe about the runtime of the two implementations as T and H increase? Explain your finding.

已思考（用时 23 秒）

> We are given three graphs showing runtime vs T for different values of H (H=2, H=8, H=32). The graphs compare the recursive unrolled implementation and the convolution-based implementation.

Observations:

1. For small H (H=2), the unrolled implementation is faster for small T, but as T increases, the convolution-based implementation becomes faster.
   The convolution-based implementation has a higher overhead (due to building the kernel and performing the convolution) but scales better with T.
2. For H=8, we see a similar trend: the unrolled implementation is faster for very small T, but the convolution-based implementation becomes faster as T increases.
3. For H=32, the unrolled implementation is slower than the convolution-based implementation even for small T. The convolution-based implementation scales better with T.

Explanation:

- The unrolled implementation has a time complexity of O(T * H^2) because at each time step we do a matrix-vector multiplication (which is O(H^2) for each sample in the batch, but note the batch is fixed and we are looking at T and H). The recurrence is:
  h_{t+1} = W h_t + ... [W is HxH, h_t is H]
  and we do this for T steps.
- The convolution-based implementation has two parts:
  a. Building the kernel: This is done recursively and requires matrix multiplications for powers of W. However, note that the kernel is built once and then used for the entire batch.
  b. The convolution operation: The convolution is done using F.conv1d, which is highly optimized and uses FFT or direct convolution algorithms. The complexity of the convolution is O(T log T) or O(T) depending on the implementation, but importantly it does not scale with H in the same way as the unrolled version.
  However, note that the kernel has shape (H, H, T) and the convolution is done over T time steps. The complexity of the convolution is O(H^2 * T) if using direct convolution, but with FFT it could be O(H^2 * T log T). However, in practice, the conv1d function in PyTorch is very efficient.
  The main point is that the convolution-based implementation can leverage parallel computation and optimized convolution routines, while the unrolled implementation is sequential.
- As H increases, the unrolled implementation becomes more expensive because of the O(H^2) operations per time step. On the other hand, the convolution-based implementation also has O(H^2) operations but they are done in parallel (in the convolution). Additionally, the convolution operation is optimized in PyTorch.
- As T increases, the unrolled implementation scales linearly with T, while the convolution-based implementation scales with T (or T log T) but with a smaller constant factor due to optimization.

Therefore, for small T and small H, the overhead of building the kernel and setting up the convolution may make the convolution-based implementation slower. But for larger T and/or

> H, the convolution-based implementation is faster.
>
> Let me summarize:
>
> - For fixed H, as T increases, the convolution-based implementation becomes more efficient than the unrolled one.
> - For fixed T, as H increases, the convolution-based implementation becomes more efficient than the unrolled one.
>
> This is because the unrolled implementation is O(T * H^2) with a sequential process, while the convolution-based implementation uses parallelizable operations and optimized convolution routines.

Based on the runtime graphs comparing the unrolled (recursive) and convolution-based implementations for different hidden dimensions H (2, 8, 32) as T increases, here are my observations:

Key Observations:

1. Dependence on Sequence Length T:

- **For small T**: The unrolled implementation is faster because it has lower overhead. It simply loops through T steps with simple matrix multiplications.
- **For large T**: The convolution-based implementation becomes significantly faster. This is expected because:
  - Unrolled implementation: **O(T)** time complexity (linear in T)
  - Convolution implementation: **O(T log T)** or **O(T)** with efficient FFT-based convolution, which has better asymptotic scaling

2. Dependence on Hidden Dimension H:

H = 2 (small hidden dimension):

- The crossover point where convolution becomes faster occurs at larger T
- For small hidden dimensions, the overhead of building the convolution kernel and performing the convolution is relatively high compared to the simple sequential updates

H = 8 (medium hidden dimension):

- Convolution becomes faster at a moderate T

- The benefits of parallel computation in convolution start to outweigh the overhead at smaller sequence lengths

H = 32 (larger hidden dimension):

- Convolution is faster even for relatively small T
- With larger H, the matrix multiplications in the unrolled implementation ($O(H^2)$ per time step) become expensive
- The convolution implementation can leverage optimized BLAS operations and parallelism more effectively

Explanation of Findings:

Computational Complexity:

- **Unrolled (recursive) implementation**:
  - Time: **$O(T \times H^2)$** (matrix-vector multiplication at each step)
  - Memory: **$O(H)$** for current state (plus $O(T \times H)$ if storing all states)
  - Sequential dependencies prevent parallelization across time
- **Convolution-based implementation**:
  - Time: **$O(H^2 \times T)$** for direct convolution, potentially **$O(H^2 \times T \log T)$** with FFT
  - Memory: **$O(H^2 \times T)$** for storing the kernel
  - Can be parallelized across time using efficient convolution algorithms

Why the Trends Occur:

1. **Fixed Overhead**: The convolution implementation has fixed overhead for:
   - Building the kernel (especially with divide-and-conquer recursion)
   - Setting up the convolution operation
   - Padding and memory allocation
2. **Parallelization Benefits**: As H increases, the convolution implementation benefits more from:
   - Matrix-matrix multiplications (more efficient than matrix-vector in the unrolled version)

- Optimized BLAS libraries
- Parallel computation across the hidden dimension

3. **Asymptotic Behavior**: For very long sequences (large T), the convolution's better asymptotic scaling dominates
4. **Memory Hierarchy**: Convolution operations are more cache-friendly and can better utilize modern CPU/GPU memory hierarchies

Practical Implications:

1. **Short sequences with small H**: Use unrolled implementation (lower overhead)
2. **Long sequences or large H**: Use convolution implementation (better scaling)
3. **Real-time applications**: Unrolled might be better for streaming with strict latency requirements (no precomputation of kernel)
4. **Batch processing**: Convolution excels when processing multiple sequences simultaneously (batch dimension N)

This trade-off is exactly why modern SSM architectures like S4 and Mamba use different strategies: some use convolution for training efficiency and switch to recurrent form for efficient inference.

> Ok now we look at another problem in a new noyebook, which is simillar to the previous one.:Introduction
> In this notebook, we'll implement implement the forward pass of an SSM (State Space Model) using recursion and convolution based approaches. We'll also compare the two approaches in terms of speed and memory usage.
>
> You will need GPU for this notebook which can be enabled via changing the runtime.
>
> You can copy your solutions from the q_coding_ssm_forward_cpu notebook for the first part of the notebook. import time
> import math
> import matplotlib.pyplot as plt
>
> import torch
> import numpy as np
> import torch.nn.functional as F

torch.set_default_device('cuda') SSM Update Rule
We consider an RNN described by the update:

$h_{t+1} = W h_t + U x_t + b$

for $t = 0, 1, \ldots, T-1$. The variables are:

$h_t \in \mathbb{R}^H$, the hidden state at time t.
$x_t \in \mathbb{R}^{N \times D}$, the input at time t.
$W \in \mathbb{R}^{H \times H}$, the recurrent weight matrix.
$U \in \mathbb{R}^{H \times D}$, the input projection matrix.
$b \in \mathbb{R}^H$, the bias vector.
N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H. Below you will implement the forward pass for the SSM using recursion based approach. The unrolled_ssm_forward function will take weights W, U, b and input x and return the hidden states h across different time steps. def unrolled_ssm_forward(W, U, b, x):

```
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to h_{t+1} in the usual notation).
    """

    ##############################################################################
    #                 TODO: Implement the recurrent pass here                    #
    ##############################################################################

    ##############################################################################
    ##############################################################################


    ##############################################################################
```

```
##############################
#                END OF YOUR CODE                #

##########################################################
##############################
```

return h_all So complete the TODO for the first question

已思考（用时 2 秒）

> We are on GPU now, but the implementation is the same as before.
> We need to make sure that the tensors are on the same device (cuda).
> However, the function should work as before, but we are using GPU for acceleration.
>
> Let's copy the recursive implementation from the previous notebook.

Here's the implementation of `unrolled_ssm_forward` for the GPU version. The logic is the same as before, but we'll ensure it works correctly on GPU:

python

```python
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
             (h_all[t] corresponds to h_{t+1} in the usual notation).
    """
    ##############################################################################
    #                  TODO: Implement the recurrent pass here                   #
    ##############################################################################

    N, T, D = x.shape
    H = W.shape[0]

    # Initialize output tensor on the same device as input
    h_all = torch.zeros(N, T, H, device=x.device, dtype=x.dtype)

    # Initialize current hidden state to zeros
    h_current = torch.zeros(N, H, device=x.device, dtype=x.dtype)
```

Comment: Although the reasoning is strong, DeepSeek sometimes repeats explanations or includes multiple alternative implementations. A more condensed answer might be preferable.

```
    # Iterate through time steps
    for t in range(T):
        x_t = x[:, t, :]  # (N, D)

        # Compute next hidden state: h_{t+1} = W h_t + U x_t + b
        h_next = torch.matmul(h_current, W.T) + torch.matmul(x_t, U.T) + b

        # Store the result
        h_all[:, t, :] = h_next

        # Update current hidden state for next iteration
        h_current = h_next

    ##############################################################################
    #                                END OF YOUR CODE                            #
    ##############################################################################
    return h_all
```

Key points for GPU compatibility:

1. **Device awareness**: The function automatically uses the same device as the input `x` through:
   - `device=x.device` when creating tensors
   - This ensures compatibility whether we're on CPU or GPU
2. **Data type consistency**: Using `dtype=x.dtype` ensures the output has the same precision (float32, float64, etc.) as the input
3. **GPU efficiency**: The implementation is identical to the CPU version because PyTorch automatically handles GPU operations. The only difference is that tensors are created on the GPU when `x` is on GPU.

Alternative implementation with explicit device handling:

```
python

def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
```

Comment: The model automatically adapts code to CUDA by using device=x.device, avoiding common GPU-CPU mismatch bugs. Good understanding of PyTorch device semantics.

```
    U: (H, D) input projection
    b: (H,)   bias
    x: (N, T, D) input sequence over T steps
  Returns:
    h_all: (N, T, H) hidden states for t=1..T
           (h_all[t] corresponds to h_{t+1} in the usual notation).
  """
  ##############################################################################
  #                    TODO: Implement the recurrent pass here                 #
  ##############################################################################

  # Get dimensions
  N, T, D = x.shape
  H = W.shape[0]

  # Initialize hidden states tensor on the same device as input
  h_all = torch.zeros(N, T, H, device=x.device, dtype=x.dtype)

  # Initialize hidden state at time 0
  h = torch.zeros(N, H, device=x.device, dtype=x.dtype)

  # Process each time step
  for t in range(T):
      # Current input
      x_t = x[:, t, :]

      # SSM update: h_{t+1} = W h_t + U x_t + b
      # Note: We need to handle broadcasting of b to (N, H)
      h = h @ W.T + x_t @ U.T + b

      # Store the updated hidden state
      h_all[:, t, :] = h

  ##############################################################################
  #                                END OF YOUR CODE                            #
  ##############################################################################
  return h_all
```

This implementation:

1. Uses `@` operator for matrix multiplication (cleaner syntax)
2. Properly handles broadcasting of bias `b`
3. Maintains device consistency
4. Follows the same recurrence: `h_{t+1} = W h_t + U x_t + b`

Note about indexing: As before, `h_all[:, 0, :]` corresponds to h_1 (the hidden state after processing x_0), and `h_all[:, t, :]` corresponds to `h_{t+1}`.

Good job! now complete the TODOs following for these questions:
Convolution Based Implementation
In the previous problem, you showed that the forward pass of an SSM can be implmemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that T is a power of 2.

You will implement two functions

make_conv_kernel(W, T): This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel K. Given that T is a power of 2, you can implement this using a divide and conquer based approach.
conv_ssm_forward(W, U, b, x): This function will take weights W, U, b and input x and return the hidden states h across different time steps. def make_conv_kernel(W, T):
"""
Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
the ssm forward pass using conv1d.

Args:
W: (H, H) weight matrix
T: scalar

Returns:
kernel_for_conv: (H, H, T) tensor
"""

```
##############################################################################
#                  TODO: Implement the kernel here                           #

##############################################################################


##############################################################################
#                              END OF YOUR CODE                              #
```

```
##############################################################################
  return kernel_for_conv  def conv_ssm_forward(W, U, b, x):
  """
  Convolution-based forward pass for a batch of sequences.

  RNN update:  h_{t+1} = W h_t + U x_t + b

  Args:
    W: (H, H) weight matrix
    U: (H, D) input projection
    b: (H,)   bias
    x: (N, T, D) input (batch=N, time steps=T, input dim=D)

  Returns:
    h_all: (N, T, H) hidden states
  """
  N, T, D = x.shape
  H = W.shape[0]

  s = x @ U.T + b

  s = s.permute(0, 2, 1)

  # Build the kernel with shape (H, H, T).
  kernel = make_conv_kernel(W, T)

##############################################################################
  #                    TODO: Implement the convolution here                    #

##############################################################################


##############################################################################
  #                        END OF YOUR CODE                                    #

##############################################################################
  return h_all  Sanity Check
```

We can compare the outputs of the two implementations to check if they are consistent. def sanity_check():
    T = 8   # number of time steps
    H = 4   # hidden dimension
    D = 3   # input dimension
    N = 2

    torch.manual_seed(0)

    W = torch.randn(H, H) * 0.1
    U = torch.randn(H, D) * 0.1
    b = torch.randn(H) * 0.1

    x = torch.randn(N, T, D)

    h_unrolled = unrolled_ssm_forward(W, U, b, x)
    h_conv = conv_ssm_forward(W, U, b, x)

    diff = (h_unrolled - h_conv).abs().max()
    print("Unrolled h(t):")
    print(h_unrolled)
    print("\nConv-based h(t):")
    print(h_conv)
    print("\nMax absolute difference:", diff.item())

sanity_check() Introduction
In this notebook, we'll implement implement the forward pass of an SSM (State Space Model) using recursion and convolution based approaches. We'll also compare the two approaches in terms of speed and memory usage.

You will need GPU for this notebook which can be enabled via changing the runtime.

You can copy your solutions from the q_coding_ssm_forward_cpu notebook for the first part of the notebook.

Imports

[ ]
```
import time
import math
import matplotlib.pyplot as plt
```

```
import torch
import numpy as np
import torch.nn.functional as F

torch.set_default_device('cuda')
```

SSM Update Rule

We consider an RNN described by the update:

$$h_{t+1} = Wh_t + Ux_t + b$$

for $t = 0, 1, \ldots, T-1$. The variables are:

$h_t \in \mathbb{R}^H$, the hidden state at time t.
$x_t \in \mathbb{R}^{N \times D}$, the input at time t.
$W \in \mathbb{R}^{H \times H}$, the recurrent weight matrix.
$U \in \mathbb{R}^{H \times D}$, the input projection matrix.
$b \in \mathbb{R}^H$, the bias vector.
N is the batch size, D is the input dimension, and H is the hidden state dimension. We assume $h_0 = 0$, the all-zero vector of dimension H.

Below you will implement the forward pass for the SSM using recursion based approach. The unrolled_ssm_forward function will take weights W, U, b and input x and return the hidden states h across different time steps.

[ ]

```
def unrolled_ssm_forward(W, U, b, x):
    """
    Unroll the linear RNN in time:
        h_{t+1} = W h_t + U x_t + b
    with initial h_0 = 0.

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input sequence over T steps
    Returns:
      h_all: (N, T, H) hidden states for t=1..T
            (h_all[t] corresponds to h_{t+1} in the usual notation).
```

```
    """

    ##########################################################
    ##############################
    #               TODO: Implement the recurrent pass here
    #

    ##########################################################
    ##############################


    ##########################################################
    ##############################
    #                     END OF YOUR CODE                    #

    ##########################################################
    ##############################
    return h_all
```

Convolution Based Implementation

In the previous problem, you showed that the forward pass of an SSM can be implmemented using a convolution operation. In this problem, you will implement the forward pass of an SSM using a convolution based approach. You can assume that T is a power of 2.

You will implement two functions

make_conv_kernel(W, T): This function will take the recurrent weight matrix W and the number of time steps T and return the convolution kernel K. Given that T is a power of 2, you can implement this using a divide and conquer based approach.

conv_ssm_forward(W, U, b, x): This function will take weights W, U, b and input x and return the hidden states h across different time steps.

[ ]

```
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when
implementing
    the ssm forward pass using conv1d.

    Args:
      W: (H, H) weight matrix
```

```
    T: scalar

  Returns:
    kernel_for_conv: (H, H, T) tensor
  """

##############################################################################
  #                    TODO: Implement the kernel here                         #

##############################################################################



##############################################################################
  #                          END OF YOUR CODE                                  #

##############################################################################
  return kernel_for_conv

[ ]
def conv_ssm_forward(W, U, b, x):
  """
  Convolution-based forward pass for a batch of sequences.

  RNN update:  h_{t+1} = W h_t + U x_t + b

  Args:
    W: (H, H) weight matrix
    U: (H, D) input projection
    b: (H,)   bias
    x: (N, T, D) input (batch=N, time steps=T, input dim=D)

  Returns:
    h_all: (N, T, H) hidden states
  """
  N, T, D = x.shape
  H = W.shape[0]

  s = x @ U.T + b
```

```
    s = s.permute(0, 2, 1)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)

################################################################################
    #                     TODO: Implement the convolution here                     #

################################################################################



################################################################################
    #                          END OF YOUR CODE                                    #

################################################################################
    return h_all
```

Sanity Check

We can compare the outputs of the two implementations to check if they are consistent.

[ ]

```
    b = torch.randn(H) * 0.1

    x = torch.randn(N, T, D)

    h_unrolled = unrolled_ssm_forward(W, U, b, x)
    h_conv = conv_ssm_forward(W, U, b, x)

    diff = (h_unrolled - h_conv).abs().max()
    print("Unrolled h(t):")
    print(h_unrolled)
```

Implementation Complexity

We can compare the two implementations in terms of efficiency. Particularly, we will compare the time taken by the two implementations to compute the hidden states for a given input and weights with varying number of time steps. import ipywidgets as widgets

```
from ipywidgets import interact

def measure_runtime(method_fn, W, U, b, x, warmup=1, repeats=10):
    # Warm-up runs (ignored in timing):
    for _ in range(warmup):
        method_fn(W, U, b, x)

    # Timed runs:
    start = time.time()
    for _ in range(repeats):
        method_fn(W, U, b, x)
    end = time.time()

    avg_time = (end - start) / repeats
    return avg_time


def run():
    T_values_cache = {}
    times_unrolled_vs_T_cache = {}
    times_conv_vs_T_cache = {}

    for H in [4, 8, 16, 32, 64, 128, 256, 512]:
      # We'll keep D, N fixed
      D = 32
      N = 32

      T_values = [8, 32, 128, 256, 512]

      # Build random U, b
      U = torch.randn(H, D)*0.1
      b = torch.randn(H)*0.1

      times_unrolled_vs_T = []
      times_conv_vs_T = []

      for T in T_values:

          diag_vals = torch.randn(H)*0.05
          W = torch.randn(H, H)*0.05
          x = torch.randn(N, T, D)

          t_unrolled = measure_runtime(unrolled_ssm_forward, W, U, b,
```

```
x)

        t_conv = measure_runtime(conv_ssm_forward, W, U, b, x)

        times_unrolled_vs_T.append(t_unrolled)
        times_conv_vs_T.append(t_conv)

    T_values_cache[H] = T_values
    times_unrolled_vs_T_cache[H] = times_unrolled_vs_T
    times_conv_vs_T_cache[H] = times_conv_vs_T
  return T_values_cache, times_unrolled_vs_T_cache,
times_conv_vs_T_cache

T_values_cache, times_unrolled_vs_T_cache, times_conv_vs_T_cache
= run()

@interact(H=widgets.FloatLogSlider(min=2, max=9, base=2,
value=4, step=1))
def interactive_benchmark(H):
  """
  Compare unrolled vs. diagonal-convolution RNN forward for
various T,
  at a chosen hidden dimension H from the slider.
  """
  H = int(H)
  T_values = T_values_cache[H]
  T_unrolled = times_unrolled_vs_T_cache[H]
  T_conv = times_conv_vs_T_cache[H]

  # Plot
  plt.figure(figsize=(6,4))
  plt.plot(T_values, T_unrolled, label="Unrolled", marker='o')
  plt.plot(T_values, T_conv, label="Conv", marker='s')
  plt.title(f"Runtime vs T, H={H}")
  plt.xlabel("Time Steps (T)")
  plt.ylabel("Runtime (sec)")
  plt.yscale('log')
  plt.grid(True)
  plt.legend()
  plt.show()
```

已思考（用时 130 秒） ›

Now I'll implement the convolution-based SSM forward pass for the GPU version. The logic is similar to the CPU version, but we need to ensure it works correctly on GPU:

python

```python
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using conv1d.

    Args:
      W: (H, H) weight matrix
      T: scalar

    Returns:
      kernel_for_conv: (H, H, T) tensor
    """
    ##############################################################################
    #                          TODO: Implement the kernel here                   #
    ##############################################################################

    H = W.shape[0]

    # Base case: T = 1
    if T == 1:
        # Identity matrix
        return torch.eye(H, device=W.device, dtype=W.dtype).unsqueeze(-1)  # (H, H,
1)

    # Divide: compute kernel for half the sequence
    T_half = T // 2
    K_half = make_conv_kernel(W, T_half)  # (H, H, T_half)

    # Compute W^(T_half) using repeated squaring
    W_power = torch.matrix_power(W, T_half)

    # Conquer: second half = W^(T_half) @ first half
    second_half = torch.matmul(W_power, K_half)  # (H, H, T_half)

    # Concatenate along time dimension
    kernel_for_conv = torch.cat([K_half, second_half], dim=-1)

    ##############################################################################
    #                                END OF YOUR CODE                            #
    ##############################################################################
    return kernel_for_conv
```

```python
def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  h_{t+1} = W h_t + U x_t + b

    Args:
      W: (H, H) weight matrix
      U: (H, D) input projection
      b: (H,)   bias
      x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
      h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b  # (N, T, H)

    s = s.permute(0, 2, 1)  # (N, H, T)

    # Build the kernel with shape (H, H, T).
    kernel = make_conv_kernel(W, T)

    ##############################################################################
    #                          TODO: Implement the convolution here              #
    ##############################################################################

    # Pad s on the left with (T-1) zeros for causal convolution
    s_padded = F.pad(s, (T-1, 0))  # (N, H, 2T-1)

    # We need to reverse the kernel along the time dimension for causal convolution
    # This is because conv1d performs cross-correlation, not convolution
    kernel_reversed = torch.flip(kernel, dims=[-1])  # (H, H, T)

    # Perform the convolution
    # Input: (N, H, 2T-1), Kernel: (H, H, T)
    # Output: (N, H, T) (since (2T-1) - T + 1 = T)
    h_conv = F.conv1d(s_padded, kernel_reversed, padding=0)  # (N, H, T)

    # Permute back to (N, T, H)
    h_all = h_conv.permute(0, 2, 1)  # (N, T, H)

    ##############################################################################
```

```
    #                                END OF YOUR CODE                              #
    ###############################################################################
    return h_all
```

Alternative Implementation for `make_conv_kernel` (Iterative version):

If you prefer an iterative approach instead of recursive (which can be more efficient on GPU), here's an alternative:

python

```
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using conv1d.

    Args:
      W: (H, H) weight matrix
      T: scalar

    Returns:
      kernel_for_conv: (H, H, T) tensor
    """
    ###############################################################################
    #                          TODO: Implement the kernel here                    #
    ###############################################################################

    H = W.shape[0]

    # Pre-allocate kernel
    kernel = torch.zeros(H, H, T, device=W.device, dtype=W.dtype)

    # First element: identity matrix
    kernel[:, :, 0] = torch.eye(H, device=W.device, dtype=W.dtype)

    # Compute powers of W iteratively
    W_power = torch.eye(H, device=W.device, dtype=W.dtype)
    for t in range(1, T):
        W_power = W_power @ W  # W^t
        kernel[:, :, t] = W_power

    ###############################################################################
    #                                END OF YOUR CODE                              #
```

```
    ################################################################################
    return kernel
```

Explanation:

1. `make_conv_kernel`:

- **Recursive version**: Uses divide-and-conquer to efficiently compute `[I, W, W², ..., W^{T-1}]`
- **Iterative version**: Simpler but may be slower for large T due to sequential computation
- **GPU considerations**: Both versions automatically use the same device as `W` (GPU if `W` is on GPU)

2. `conv_ssm_forward`:

1. **Compute** `s_t = U x_t + b` for all time steps
2. **Transpose** to `(N, H, T)` for convolution
3. **Pad input** with `T-1` zeros on the left for causality
4. **Reverse kernel** along time dimension because `F.conv1d` performs cross-correlation, not true convolution
5. **Perform convolution** to get `(N, H, T)` output
6. **Transpose back** to `(N, T, H)`

Why we reverse the kernel:

The SSM recurrence gives: `h_{t+1} = Σ_{k=0}^{t} W^{t-k} s_k`
This is equivalent to convolution with kernel `[W^{T-1}, W^{T-2}, ..., W, I]` applied to `s`.
Since `F.conv1d` does cross-correlation (no kernel flipping), we need to pre-reverse the kernel.

Mathematical equivalence:

- For T=3, we want: $h_3 = W^2 s_0 + W s_1 + s_2$
- Kernel should be: `[W², W, I]`

- With input $[s_0, s_1, s_2]$ padded as $[0, 0, s_0, s_1, s_2]$
- `conv1d` at position 2 gives: $W^2*s_0 + W*s_1 + I*s_2 = h_3$

Complete code for GPU:

Here's the complete implementation that should work correctly on GPU:

python

```python
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using conv1d.
    """
    H = W.shape[0]

    if T == 1:
        return torch.eye(H, device=W.device, dtype=W.dtype).unsqueeze(-1)

    T_half = T // 2
    K_half = make_conv_kernel(W, T_half)

    # Use matrix_power for efficient exponentiation
    W_power = torch.matrix_power(W, T_half)

    second_half = torch.matmul(W_power, K_half)

    return torch.cat([K_half, second_half], dim=-1)


def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b  # (N, T, H)
    s = s.permute(0, 2, 1)  # (N, H, T)

    kernel = make_conv_kernel(W, T)  # (H, H, T)

    # Pad for causal convolution
    s_padded = F.pad(s, (T-1, 0))  # (N, H, 2T-1)

    # Reverse kernel for correct convolution
```

```
    kernel_reversed = torch.flip(kernel, dims=[-1])

    # Perform convolution
    h_conv = F.conv1d(s_padded, kernel_reversed, padding=0)  # (N, H, T)

    # Transpose back
    return h_conv.permute(0, 2, 1)  # (N, T, H)
```

These implementations should work correctly on GPU and pass the sanity check.

> Now complete these TODOs: Introducing structure: Diagonal Weight Matrics
> We can optimize the convolution implemntation via adding a constraint on the W matrix ensuring it is diagonal. We can make the convolution implementation more efficient by leveraging depthwise convolutions for implementing the forward operation. def make_diag_depthwise_kernel(W, T):
> """
> Construct a depthwise 1D conv kernel for W with shape (H,H). W is a diagonal matrix.
>
> Args:
> W: (H, H) diagonal matrix
> T: (int) number of time steps
>
> Return:
> kernel of shape (H, 1, T), which can be used in depthwise convolution
> """
>
> ##
> # TODO: Implement the kernel here #
> ##
>
>
> ##
> # END OF YOUR CODE #
> ##

```
############################
  return kernel  def diag_conv_ssm_forward(W, U, b, x):
  """
  Convolution-based forward pass for an RNN with W

  RNN update:  h_{t+1} = W h_t + U x_t + b
  but W is diagonal, so h_{t+1}(i) = diagW[i]*h_t(i) + [U x_t + b](i).

  Args:
    W: (H, H) [diagonal in practice]
    U: (H, D)
    b: (H,)
    x: (N, T, D) => N = batch, T = time steps, D = input dim

  Returns:
    h_all: (N, T, H)
  """
  N, T, D = x.shape
  H = W.shape[0]

  s = x @ U.T + b

  s = s.permute(0, 2, 1)  # (N,H,T)

  # Build kernel (H,1,T) for depthwise conv
  kernel = make_diag_depthwise_kernel(W, T)

##########################################################
############################
  #                 TODO: Implement the convolution here           #
  #                 Hint: Use `groups` argument                    #

##########################################################
############################


##########################################################
############################
  #                       END OF YOUR CODE                         #

##########################################################
############################
  return h_all
```

Optimizing the recurrent implementation
Similarly, we can also optimize the recurrent implementation by using the diagonal nature of the W matrix. def diag_unrolled_ssm_forward(W, U, b, x):
"""
Forward pass for:
h_{t+1} = W h_t + U x_t + b
but W is diagonal, i.e. W = diag(diagW).

Args:
W: (H, H) diagonal matrix
U: (H, D)
b: (H,)
x: (N, T, D) => batch=N, time steps=T, input dim=D

Returns:
h_all: (N, T, H)
"""

##
TODO: Implement the optimized recurrent pass here
#

##

##
END OF YOUR CODE
##
return h_all

已思考（用时 16 秒）

We are given that W is a diagonal matrix. We can optimize both the convolution and recurrent implementations by leveraging this structure.

1. For the convolution kernel (diagonal case):
   Instead of building a full (H, H, T) kernel, we can build a (H, 1, T) kernel for depthwise convolution.
   The kernel for the i-th channel should be [1, w_i, w_i^2, ..., w_i^{T-1}] where w_i = W[i,i].
2. For the convolution forward pass:
   We can use a depthwise convolution with groups=H and kernel of shape (H, 1, T).
3. For the recurrent forward pass (diagonal case):
   We can update each hidden dimension independently because the update for the i-th dimension is:
   h_{t+1, i} = w_i * h_{t, i} + [U x_t + b]_i
   This can be done efficiently by extracting the diagonal of W and then doing element-wise operations.

Let's implement:

make_diag_depthwise_kernel:
Input: W (H, H) [diagonal], T
Output: kernel (H, 1, T)

diag_conv_ssm_forward:
We'll use F.conv1d with groups=H for depthwise convolution.

diag_unrolled_ssm_forward:
We can extract the diagonal of W and then for each time step, do:
h_{next} = diagW * h_current + s_t
where s_t = x_t @ U.T + b.

Implementation:

Step 1: In make_diag_depthwise_kernel, we assume W is diagonal. We can take its diagonal and then for each i, compute the powers up to T-1.

Step 2: In diag_conv_ssm_forward, we use the kernel in a depthwise convolution. Note: we need to pad and reverse the kernel appropriately.

Step 3: In diag_unrolled_ssm_forward, we use the diagonal of W to update the hidden state with element-wise multiplication.

However, note that the provided W is still a full matrix, but we are told it is diagonal. We can extract the diagonal elements.

Let's code accordingly.

Now I'll implement the diagonal versions of the SSM forward pass, which are more efficient since `w` is diagonal. This means each hidden dimension evolves independently:

```
python
```

```
def make_diag_depthwise_kernel(W, T):
    """
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is a diagonal matr
ix.

    Args:
      W: (H, H) diagonal matrix
      T: (int) number of time steps

    Return:
      kernel of shape (H, 1, T), which can be used in depthwise convolution
    """
    ##############################################################################
    #                           TODO: Implement the kernel here                  #
    ##############################################################################

    H = W.shape[0]

    # Extract the diagonal elements of W
    # For a diagonal matrix, W[i, i] are the diagonal values
    # We can use torch.diag() to extract the diagonal
    diag_vals = torch.diag(W)  # (H,)

    # Initialize kernel tensor
    kernel = torch.zeros(H, 1, T, device=W.device, dtype=W.dtype)

    # For each hidden dimension, compute powers of the diagonal element
    # Kernel for dimension i: [1, w_i, w_i^2, ..., w_i^{T-1}]
    for i in range(H):
        w = diag_vals[i]
        # Compute powers: w^0, w^1, w^2, ..., w^{T-1}
        for t in range(T):
            kernel[i, 0, t] = w ** t

    ##############################################################################
    #                                END OF YOUR CODE                            #
    ##############################################################################
    return kernel


def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with W

    RNN update:  h_{t+1} = W h_t + U x_t + b
    but W is diagonal, so h_{t+1}(i) = diagW[i]*h_t(i) + [U x_t + b](i).
```

```
    Args:
      W: (H, H) [diagonal in practice]
      U: (H, D)
      b: (H,)
      x: (N, T, D) => N = batch, T = time steps, D = input dim

    Returns:
      h_all: (N, T, H)
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b  # (N, T, H)
    s = s.permute(0, 2, 1)  # (N, H, T)

    # Build kernel (H,1,T) for depthwise conv
    kernel = make_diag_depthwise_kernel(W, T)

    ##############################################################################
    #                          TODO: Implement the convolution here              #
    #                          Hint: Use `groups` argument                       #
    ##############################################################################

    # Pad s on the left with (T-1) zeros for causal convolution
    s_padded = F.pad(s, (T-1, 0))  # (N, H, 2T-1)

    # Reverse kernel for correct convolution
    kernel_reversed = torch.flip(kernel, dims=[-1])  # (H, 1, T)

    # Perform depthwise convolution with groups=H
    # Input: (N, H, 2T-1), Kernel: (H, 1, T), groups=H
    # Output: (N, H, T) (since (2T-1) - T + 1 = T)
    h_conv = F.conv1d(s_padded, kernel_reversed, groups=H, padding=0)  # (N, H, T)

    # Permute back to (N, T, H)
    h_all = h_conv.permute(0, 2, 1)  # (N, T, H)

    ##############################################################################
    #                                END OF YOUR CODE                            #
    ##############################################################################
    return h_all


def diag_unrolled_ssm_forward(W, U, b, x):
    """
    Forward pass for:
```

```
        h_{t+1} = W h_t + U x_t + b
    but W is diagonal, i.e. W = diag(diagW).

    Args:
      W: (H, H) diagonal matrix
      U: (H, D)
      b: (H,)
      x: (N, T, D)  => batch=N, time steps=T, input dim=D

    Returns:
      h_all: (N, T, H)
    """
    ##############################################################################
    #             TODO: Implement the optimized recurrent pass here              #
    ##############################################################################

    N, T, D = x.shape
    H = W.shape[0]

    # Extract diagonal elements of W
    # For diagonal matrix, we can extract the diagonal once and use element-wise oper
ations
    diag_vals = torch.diag(W)  # (H,)

    # Precompute s_t = U x_t + b for all t
    s = x @ U.T + b  # (N, T, H)

    # Initialize output tensor
    h_all = torch.zeros(N, T, H, device=x.device, dtype=x.dtype)

    # Initialize hidden state
    h = torch.zeros(N, H, device=x.device, dtype=x.dtype)

    # Iterate through time steps
    for t in range(T):
        # For diagonal W, we can use element-wise multiplication instead of matrix mu
ltiplication
        # h_{t+1} = diag(W) * h_t + s_t
        h = diag_vals * h + s[:, t, :]  # (N, H)

        # Store result
        h_all[:, t, :] = h

    ##############################################################################
    #                               END OF YOUR CODE                             #
```

```
    ##############################################################################
    return h_all
```

Alternative, more efficient implementation for `make_diag_depthwise_kernel`:

The nested loop implementation above is clear but not very efficient for large H and T. Here's a vectorized version:

python

```python
def make_diag_depthwise_kernel(W, T):
    """
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is a diagonal matr
ix.

    Args:
      W: (H, H) diagonal matrix
      T: (int) number of time steps

    Return:
      kernel of shape (H, 1, T), which can be used in depthwise convolution
    """
    ##############################################################################
    #                          TODO: Implement the kernel here                   #
    ##############################################################################

    H = W.shape[0]

    # Extract the diagonal elements
    diag_vals = torch.diag(W)  # (H,)

    # Create a tensor of powers: [0, 1, 2, ..., T-1]
    powers = torch.arange(T, device=W.device, dtype=W.dtype)  # (T,)

    # Compute w_i^t for all i and t
    # Shape: (H, T) where element (i, t) = diag_vals[i]^t
    # We need to broadcast: diag_vals[:, None] is (H, 1) and powers is (T,)
    # diag_vals[:, None] ** powers[None, :] gives (H, T)
    kernel_2d = diag_vals[:, None] ** powers[None, :]  # (H, T)

    # Reshape to (H, 1, T) for depthwise convolution
    kernel = kernel_2d.unsqueeze(1)  # (H, 1, T)

    ##############################################################################
```

```
#                                END OF YOUR CODE                              #
##############################################################################
return kernel
```

Explanation of the diagonal implementations:

1. `make_diag_depthwise_kernel`:

- For a diagonal matrix `W`, the diagonal elements `w_i = W[i,i]`
- The convolution kernel for dimension `i` is
  `[w_i^0, w_i^1, w_i^2, ..., w_i^{T-1}] = [1, w_i, w_i^2, ...]`
- We build a tensor of shape `(H, 1, T)` where each channel has the powers of the corresponding diagonal element

2. `diag_conv_ssm_forward`:

- Uses **depthwise convolution** with `groups=H`
- Each channel (hidden dimension) is processed independently with its own 1D kernel
- Much more efficient than full convolution when `W` is diagonal

3. `diag_unrolled_ssm_forward`:

- Since `W` is diagonal, `W h_t` is just element-wise multiplication: `diag(W) * h_t`
- This reduces computation from $O(H^2)$ to $O(H)$ per time step
- We precompute `s_t = U x_t + b` for all `t` to avoid recomputation

Why these optimizations are significant:

1. **Memory efficiency**:
   - Full kernel: `(H, H, T)` elements
   - Diagonal kernel: `(H, 1, T)` elements (H times smaller)
2. **Computational efficiency**:
   - Full convolution: $O(H^2 \times T)$ operations
   - Depthwise convolution: $O(H \times T)$ operations

3. **Recurrent optimization**:
    - Full recurrence: O(H^2) per time step
    - Diagonal recurrence: O(H) per time step

Complete optimized implementations:

Here are the final optimized versions:

```python
def make_diag_depthwise_kernel(W, T):
    """
    Construct a depthwise 1D conv kernel for W with shape (H,H). W is a diagonal matr
ix.
    """
    H = W.shape[0]

    # Extract diagonal and compute powers efficiently
    diag_vals = torch.diag(W)  # (H,)
    powers = torch.arange(T, device=W.device, dtype=W.dtype)  # (T,)

    # Compute all powers at once: (H, T)
    kernel_2d = diag_vals[:, None] ** powers[None, :]

    # Reshape for depthwise convolution: (H, 1, T)
    return kernel_2d.unsqueeze(1)


def diag_conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for an RNN with diagonal W.
    """
    N, T, D = x.shape
    H = W.shape[0]

    s = x @ U.T + b  # (N, T, H)
    s = s.permute(0, 2, 1)  # (N, H, T)

    # Build kernel (H, 1, T) for depthwise conv
    kernel = make_diag_depthwise_kernel(W, T)

    # Pad for causal convolution
    s_padded = F.pad(s, (T-1, 0))  # (N, H, 2T-1)

    # Reverse kernel for correct convolution
```

```
kernel_reversed = torch.flip(kernel, dims=[-1])

# Depthwise convolution (each channel independently)
h_conv = F.conv1d(s_padded, kernel_reversed, groups=H, padding=0)  # (N, H, T)

# Transpose back
```